ICC-133 Fundamentos de Sistemas Computacionales

Pontificia Universidad Católica Madre y Maestra

16 de julio de 2026

1. Examen Final

Base de datos de estadísticas de los paises.

1.1. Ejercicio propuesto

1. Implemente una base de datos (lista doblemente enlazada) con datos estadísticos de los diferentes países del mundo. Los datos estadísticos de cada país están disponibles en un archivo de valores separados por comas (CSV), se deberá implementar el código para leer el archivo y proceder a crear una lista con nodos doblemente enlazados en donde cada nodo corresponda a un país.

   Utilice el siguiente código mostrado como base para su proyecto.

```
typedef struct {
  char *country;           //  Country
  char *region;            //  Region
  unsigned int population; //  Population
  float area;              //  Area (sq. mi.)
  float pop_density;        //  Pop. Density (per sq. mi.)
  float coastline;      //  Coastline (coast/area ratio)
  float net_migration;    //  Net migration
  float infant_mortality;   //  Infant mortality (per 1000 births)
  int GDP;          //  GDP (US$ per capita)
  float literacy;       //  Literacy (%)
  float phones;       //  Phones (per 1000)
  float arable;       //  Arable (%)
  float crops;        //  Crops (%)
  float other;        //  Other (%)
  float climate;        //  Climate
  float birthrate;      //  Birthrate
  float deathrate;      //  Deathrate
  float agriculture;      //  Agriculture
  float industry;       //  Industry
  float service;        //  Service
} country;

typedef struct Node {
  country *coutry;
  struct Node* next;
  struct Node* prev;
} Node;

typedef enum Fields {
  NAME, POPULATION, AREA, POP_DENSITY
} Fields;

```

```
// Prototipos de las funciones
void insertAtEnd(Node* head, country ctry);
void insertAtFront(Node* head, country ctry);
country* findCountryByName(Node* head, char * name);
unsigned int countNodes(Node* head);
double findGDPAverage(Node* head);
void deleteByName(Node* head, char* name);
bool deleteFirstNode(Node* head);
bool deleteLastNode(Node* head);
country* listToArray(Node* head);
void sortLinkedList(Node* head, Fields field);
void display(Node* head);

int main(void) {

   // ******************************
   // Inserte el codigo aqui para
   // implementar y probar la lista
   // ******************************

   return 0;
}
```

Una vez estén cargados todos los nodos de la lista, para su manejo se debe de implementar las siguientes funciones:

Función	Descripción
void insertAtEnd(Node* head, country ctry)	Esta función insertará un nuevo nodo al final de lista enlazada.
void insertAtFront(Node* head, country ctry)	Esta función insertará un nuevo nodo al principio (head) de lista enlazada.
country* findCountryByName(Node* head, char * name)	Esta función devolverá un puntero al país (struct country *) que conincida con el nombre buscado.
unsigned int countNodes(Node* head)	Esta función debe retornar el total de nodos (paises) en la lista.
double findGDPAverage(Node* head)	Esta función devolverá el promedio del GDP (Gross Domestic Product o Producto Interno Bruto) de todos los países en la lista.
void deleteByName(Node* head, char* name)	Esta función permite borrar el nodo (país) de la lista que coincida con el nombre provisto.
int deleteFirstNode(Node* head)	Esta función permite borrar el primer nodo (head) de la lista.
int deleteLastNode(Node* head)	Esta función permite borrar el último nodo de la lista.
country* listToArray(Node* head)	Esta función permite crear un arreglo estático de los países a partir de la lista.
void sortLinkedList(Node* head, Fields field)	Esta función permitirá ordenar los nodos de la lista por el campo indicado.
void display(Node* head)	Esta función imprime un listado de los países en donde se muestra el nombre, población y GDP de cada país, como se muestra en el siguiente ejemplo:

NOTAS :

- Debe de implementar el código para leer el archivo CSV (**countries_of_the_world.csv**) y luego crear una base de datos (lista doblemente enlazada) que contenga cada uno delos paises y sus datos estadísticos.
- Observe que en el código se utilizan punteros de diferentes tipos ((**Node**$*$, **char**$*$, **etc**.) por lo que tendrá que utilizar manejo de memoria dinámica (**malloc**(), **free**(), etc.) para poder almacenar en memoria cada entrada para estos punteros.